



PUNJAB UNIVERSITY COLLEGE OF INFORMATION
AND TECHNOLOGY (PUCIT)

THREAT MODELLING ASSIGNMENT

Information Security

Student Name	Khadija Amer
Roll Number	BCSF24A030
Submitted To	Sir Shehryar Raza
Course	Information Security
Assignment	Threat Modelling
Submission Date	27 May 2026

Table of Contents

1. Introduction

2. Scenario 1 (Easy): User Authentication System

2.1 System Overview

2.2 Data Flow Diagram Description

2.3 Identified Threats (STRIDE)

3. Scenario 2 (Medium): Third-Party API Integration

3.1 System Overview

3.2 Data Flow Diagram Description

3.3 Identified Threats (STRIDE)

3.4 Trust Boundary Violation Analysis

4. Scenario 3 (Hard): Microservices-Based Healthcare Application

4.1 System Overview

4.2 Data Flow Diagram — Trust Boundaries

4.3 Identified Threats

4.4 Attack Chain Analysis

4.5 Prioritised Remediation Plan

5. Conclusion

1. Introduction

Threat modelling is a proactive and structured methodology used to identify, categorise, and mitigate potential security threats within a system before they can be exploited in production environments. Rather than reacting to vulnerabilities after the fact, threat modelling empowers developers and security engineers to reason about system design from an adversarial perspective, enabling the anticipation and prevention of security failures at the design stage.

In this assignment, the STRIDE threat modelling framework is applied across three progressively complex scenarios using OWASP Threat Dragon, an open-source diagrammatic threat modelling tool. STRIDE stands for Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege — providing a systematic lens through which threats can be discovered and categorised. Data Flow Diagrams (DFDs) are constructed for each scenario to represent the movement of data between system components, enabling identification of trust boundaries where adversarial manipulation is most likely to occur.

The three scenarios addressed in this report span a basic user authentication system, a third-party payment API integration in an e-commerce platform, and a complex microservices-based healthcare application. Each scenario builds upon the previous in terms of architectural complexity and the breadth of threats considered.

2. Scenario 1 (Easy): User Authentication System

2.1 System Overview

This scenario models a simple web application allowing registered users to log in using a username and password. Credentials are stored in a relational database, sessions are managed via tokens, and all communication occurs over HTTPS. Multi-factor authentication (MFA) is absent from the current design.

2.2 Data Flow Diagram Description

The DFD consists of four core components connected by data flows, separated by a trust boundary between the external user and the internal application infrastructure.

Component	Type	Description
User	External Entity	The individual attempting to authenticate to the web application.
Web Server	Process	Receives login requests, validates credentials, and manages session creation.
Session Manager	Process	Issues and validates session tokens for authenticated requests.
User Database	Data Store	Stores hashed user credentials and account information.

Data flows include: the User submitting credentials to the Web Server; the Web Server querying the User Database for credential verification; the Web Server invoking the Session Manager upon successful authentication; and the Session Manager returning a token delivered to the User. A trust boundary separates the User (external) from all internal components.

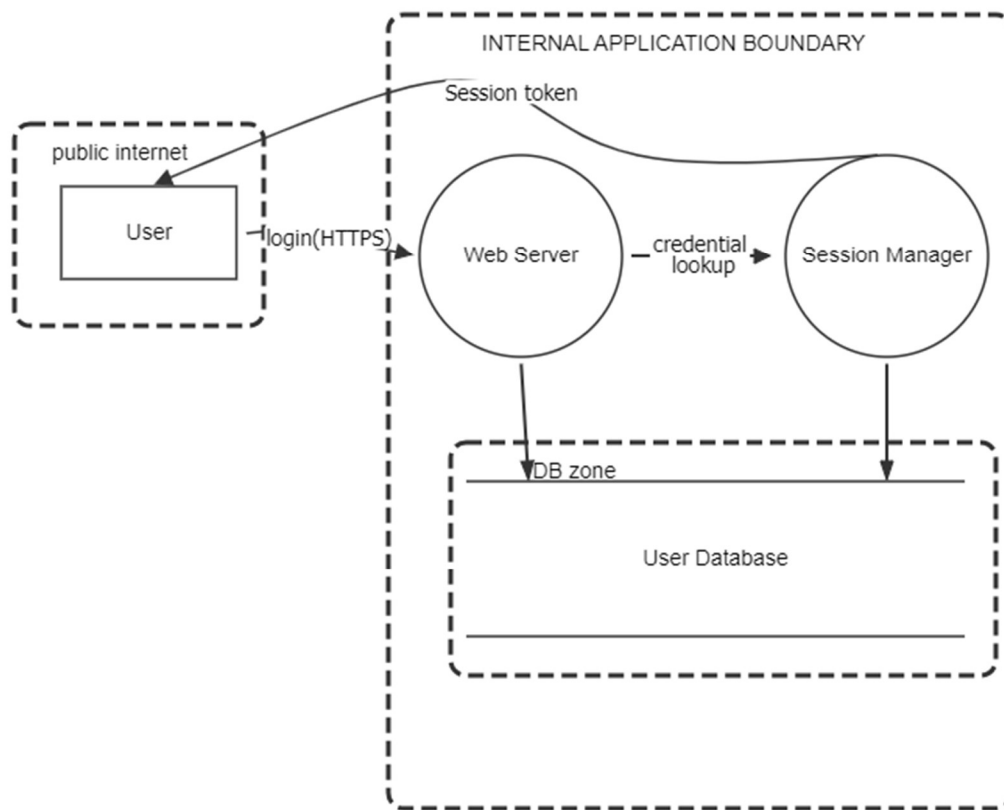


FIGURE 01: User Authentication

2.3 Identified Threats (STRIDE)

Threat 1 — Credential Spoofing (Spoofing)

STRIDE Category: Spoofing

Description: Without multi-factor authentication, an attacker who has obtained a valid username and password through phishing, credential stuffing, or a prior data breach can impersonate a legitimate user. Since the system relies solely on a single shared secret, possession of the password alone grants full account access.

Mitigation: Implement MFA for all accounts using TOTP or SMS codes. Enforce account lockout after a defined number of consecutive failed attempts to limit automated credential stuffing.

Threat 2 — Session Token Tampering (Tampering)**STRIDE Category:** Tampering

Description: If session tokens are not cryptographically signed or are predictably generated, an attacker can forge or modify a token to impersonate another user or gain elevated privileges — for example by decoding a base64 user ID, altering it, and re-encoding the modified token.

Mitigation: Generate tokens using a CSPRNG with sufficient entropy. Sign JWTs with HMAC-SHA256 or RS256 and validate the signature on every request. Store only an opaque server-side reference rather than embedding user data in the token.

Threat 3 — Credential Leak from Database (Information Disclosure)**STRIDE Category:** Information Disclosure

Description: If the database is breached and passwords are stored in plaintext or with weak hashing (MD5, SHA-1), an attacker gains immediate access to all credentials. Credential reuse across services amplifies the breach's blast radius.

Mitigation: Hash all passwords using bcrypt, scrypt, or Argon2 before storage. Restrict database access to the web server process only through network and application-level access controls.

Threat 4 — Login Endpoint Flooding (Denial of Service)**STRIDE Category:** Denial of Service

Description: An attacker can send high volumes of login requests to exhaust server resources or systematically guess credentials. Without rate limiting, this proceeds unimpeded and causes service degradation for legitimate users.

Mitigation: Implement rate limiting at the web server or reverse proxy, restricting attempts per IP per time window. Introduce CAPTCHA challenges after threshold failures. Deploy a WAF to detect volumetric authentication abuse.

3. Scenario 2 (Medium): Third-Party API Integration

3.1 System Overview

This scenario models an e-commerce platform integrating with a third-party payment gateway via a REST API secured by API keys. Transaction results are stored in an internal database and surfaced to customers via the frontend. Order confirmation notifications are dispatched through a third-party email service.

3.2 Data Flow Diagram Description

Component	Type	Description
Customer	External Entity	End user who initiates purchases and receives email confirmations.
Frontend	Process	Renders the shopping interface and collects payment initiation.
Backend Server	Process	Processes transactions, calls the payment gateway, stores results, and triggers emails.
Transaction DB	Data Store	Stores transaction records, order status, and purchase history.
Payment Gateway	External Entity	Third-party service processing card transactions via REST API.
Email Service	External Entity	Third-party SMTP/API service for sending order confirmations.

Trust Boundary 1 separates the Customer from the Frontend and internal systems. Trust Boundary 2 separates the Backend Server from both third-party services. Data flows include customer payment initiation to the frontend, backend calls to the payment gateway and email service, and persistence of transaction results in the internal database.

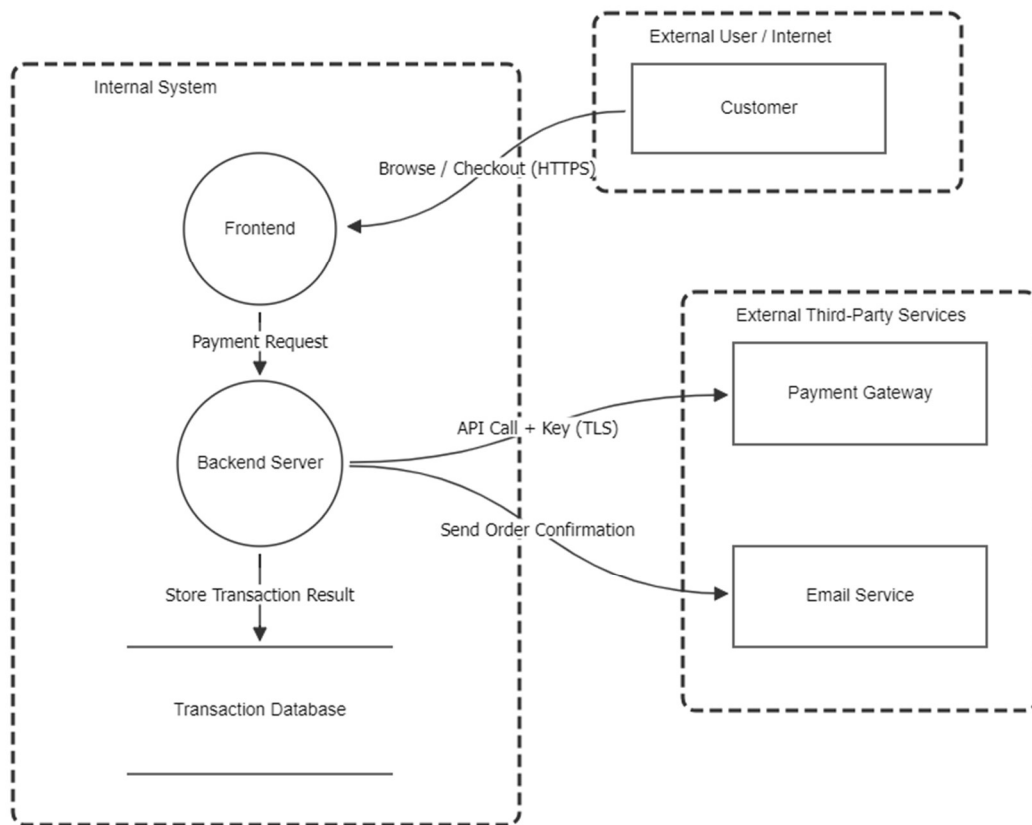


FIGURE 02 : Payment API

3.3 Identified Threats (STRIDE)

#	Threat Title	STRIDE Category	Severity
1	API Key Theft via Code Repository	Spoofing	High
2	Payment Data Tampering in Transit	Tampering	High
3	Transaction Repudiation	Repudiation	Medium
4	Card Data Exposure in Backend Logs	Information Disclosure	High

5	Email Service Abuse for Phishing	Elevation of Privilege	Medium
6	Payment Gateway Denial of Service	Denial of Service	Medium

Threat 1 — API Key Theft (Spoofing) — High

Description: Payment gateway API keys hardcoded in source code or committed environment files can be harvested by an attacker, who then impersonates the backend to initiate fraudulent transactions or harvest payment data.

Mitigation: Store all API keys in a dedicated secrets management system (e.g., HashiCorp Vault, AWS Secrets Manager). Never commit secrets to version control. Rotate keys regularly and apply IP whitelisting at the gateway.

Threat 2 — Payment Data Tampering in Transit (Tampering) — High

Description: Without certificate pinning, a MITM attacker can intercept and modify transaction data between the backend and payment gateway, altering payment amounts or destination accounts before processing.

Mitigation: Enforce mutual TLS (mTLS) and certificate pinning for all gateway communication. Validate all payment request/response fields against expected schemas and use HMAC-based message integrity checks.

Threat 3 — Transaction Repudiation (Repudiation) — Medium

Description: Without tamper-evident audit logging, users or internal actors can dispute that transactions occurred. A compromised backend could manipulate records without leaving forensic evidence, enabling fraudulent chargebacks.

Mitigation: Implement immutable, append-only audit logs for all payment events including timestamps, user IDs, amounts, and gateway responses. Use write-once storage or cryptographically chained entries. Retain logs per PCI-DSS (12 months minimum).

Threat 4 — Card Data Exposure in Logs (Information Disclosure) — High

Description: Developers may inadvertently log full payment objects including card numbers or CVV codes. If logs are exfiltrated, sensitive cardholder data is exposed, violating PCI-DSS compliance.

Mitigation: Implement log scrubbing to automatically redact sensitive fields. Adopt data minimisation policies and use tokenisation so the platform stores only gateway-issued tokens rather than raw card data.

Threat 5 — Email Service Abuse (Elevation of Privilege) — Medium

Description: An attacker gaining access to email service credentials can send fraudulent phishing emails appearing to originate from the platform domain, redirecting customers to attacker-controlled pages or delivering malware.

Mitigation: Secure email API credentials with the same rigour as payment keys. Implement DMARC, DKIM, and SPF standards to prevent domain spoofing. Monitor email sending volume for anomalous patterns.

Threat 6 — Payment Gateway DoS (Denial of Service) — Medium

Description: Without circuit breaker patterns, an attacker flooding the checkout endpoint can cause the backend to overwhelm the payment gateway, triggering rate limiting that prevents legitimate customers from completing purchases.

Mitigation: Implement circuit breaker and bulkhead patterns. Apply rate limiting on checkout endpoints per user session and IP. Use asynchronous request queuing to smooth traffic spikes before they reach the gateway.

3.4 Trust Boundary Violation Analysis

The most significant trust boundary violation occurs at the interface between the Backend Server (internal) and the Payment Gateway (external). The backend transmits API keys across this boundary with each request, over a network path not wholly under the organisation's control. If TLS is not strictly validated or the key is intercepted, an attacker gains credentials equivalent to the backend's identity from the gateway's perspective.

The security implication is that the gateway cannot distinguish the legitimate backend from an attacker in possession of a stolen key. All fraud, financial liability, and PCI-DSS consequences are attributed to the legitimate platform. This is mitigated through IP whitelisting at the gateway, short-lived tokens in place of static keys, and mTLS certificate-based authentication.

4. Scenario 3 (Hard): Microservices-Based Healthcare Application

4.1 System Overview

This scenario models a microservices-based patient management system comprising six services: Patient Portal (public-facing web frontend), API Gateway (external traffic routing), Identity Service (OAuth 2.0 / JWT issuance), Medical Records Service (PHI storage), Appointment Service (scheduling), and Notification Service (SMS/email via third-party providers). Patient data constitutes Protected Health Information (PHI) subject to HIPAA and data protection legislation.

4.2 Data Flow Diagram — Trust Boundaries

- Trust Boundary 1: Public Internet ↔ DMZ — separates users from the Patient Portal and API Gateway.
- Trust Boundary 2: DMZ ↔ Internal Services — separates the API Gateway from all backend microservices.
- Trust Boundary 3: Internal Services ↔ Each Other — separates individual microservices and third-party notification providers.

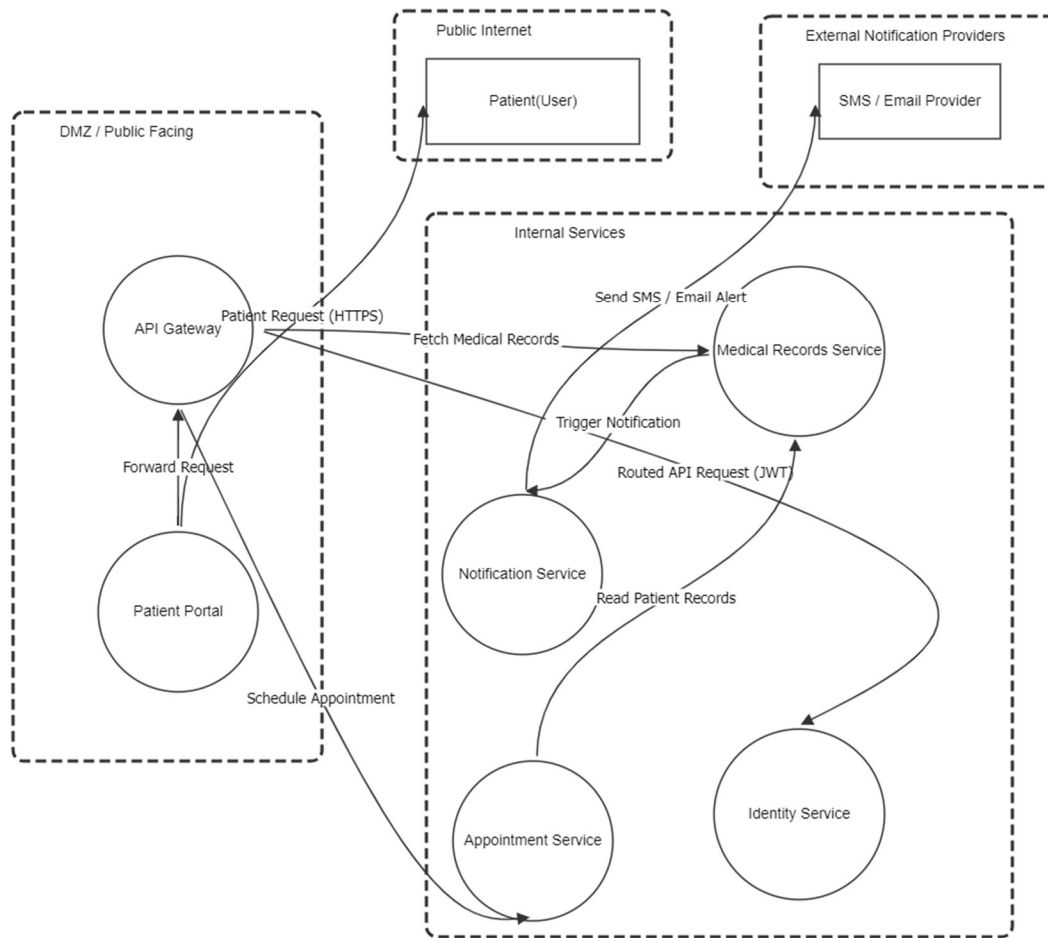


FIGURE 03 : Healthcare Microservice

4.3 Identified Threats

#	Threat Title	STRIDE	Severity	Component
1	JWT Token Forgery	Spoofing	Critical	Identity Service, API GW
2	Patient Record Tampering	Tampering	Critical	Medical Records Service
3	No Audit Trail for Record Access	Repudiation	High	Medical Records, Identity

4	JWT Signing Secret Exposure	Information Disclosure	Critical	Identity Service
5	PHI Leakage via Notification Payload	Information Disclosure	High	Notification Service
6	API Gateway DDoS	Denial of Service	High	API Gateway
7	OAuth Token Replay Attack	Spoofing	High	Identity Service, API GW
8	Appointment Data Manipulation	Tampering	Medium	Appointment Service
9	Broken Access Control on Records	Elevation of Privilege	Critical	Medical Records Service
10	SMS Provider Interception	Tampering	Medium	Notification Service

Threat 1 — JWT Token Forgery (Spoofing — Critical)

Affected Component: Identity Service, API Gateway

Description: If the JWT signing secret is weak or exposed, an attacker can forge tokens with arbitrary claims (e.g., elevated roles, arbitrary patient IDs) and gain access to any JWT-protected resource, bypassing the Identity Service entirely.

Mitigation: Use asymmetric key signing (RS256 or ES256) so the private key never leaves the Identity Service. Rotate signing keys regularly via a KMS. Validate audience, issuer, expiry, and signature on every request. (Ref: OWASP API Security Top 10, NIST 800-53 IA-5)

Threat 2 — Patient Record Tampering (Tampering — Critical)

Affected Component: Medical Records Service

Description: An attacker with write access — through a compromised service account or exploited API endpoint — could modify or delete patient records. In healthcare, this risks incorrect medication administration, surgical errors, or misdiagnosis with potentially life-threatening consequences.

Mitigation: Implement field-level integrity verification using digital signatures for sensitive fields. Apply least-privilege access — only the Appointment Service and authenticated sessions have read access, write access is strictly controlled. Maintain a cryptographically chained audit log. (Ref: HIPAA Security Rule, NIST 800-53 SI-7)

Threat 3 — No Audit Trail for Record Access (Repudiation — High)

Affected Component: Medical Records Service, Identity Service

Description: Without comprehensive audit logging of who accessed which records and when, malicious insiders or compromised accounts can access PHI without leaving a verifiable trace, undermining compliance with healthcare data regulations.

Mitigation: Implement a centralised, immutable audit logging system recording all access events: user identity, accessed resource, timestamp, and source IP. Logs must be write-protected and stored separately from the services generating them. (Ref: HIPAA Security Rule 164.312(b), ISO 27001 A.12.4)

Threat 4 — JWT Signing Secret Exposure (Information Disclosure — Critical)

Affected Component: Identity Service

Description: Exposure of the JWT signing secret through misconfigured environment variables, insecure storage, or inclusion in container images allows an attacker to forge arbitrary tokens and decrypt confidential payload data — a foundational compromise of the entire authentication architecture.

Mitigation: Store signing keys exclusively in an HSM or dedicated secrets manager (e.g., HashiCorp Vault). Never include secrets in container images, source code, or version control. Audit all secret-access operations with anomaly alerting. (Ref: NIST 800-57, OWASP Top 10 A02)

Threat 5 — PHI Leakage via Notification Payloads (Information Disclosure — High)

Affected Component: Notification Service

Description: Notification payloads containing excessive PHI (diagnosis codes, medication details) are transmitted to third-party SMS/email providers who may not be HIPAA-compliant Business Associates, constituting a regulatory violation and privacy breach.

Mitigation: Adopt data minimisation — include only appointment time and location in notifications, never clinical details. Ensure all third-party providers have signed HIPAA Business Associate Agreements (BAAs). Encrypt notification content in transit and at rest. (Ref: HIPAA Privacy Rule, GDPR Article 5(1)(c))

Threat 6 — API Gateway DDoS (Denial of Service — High)

Affected Component: API Gateway

Description: As the single entry point for all external traffic, the API Gateway is a high-value DDoS target. A successful attack renders the Patient Portal inaccessible, preventing patients from booking appointments or accessing records — with potential clinical implications in time-sensitive scenarios.

Mitigation: Deploy the API Gateway behind a cloud-based DDoS protection service (e.g., AWS Shield, Cloudflare). Implement rate limiting per client IP and per authenticated user. Use auto-scaling to absorb traffic surges. (Ref: NIST 800-53 SC-5, CIS Critical Security Controls)

Threat 7 — OAuth Token Replay Attack (Spoofing — High)

Affected Component: Identity Service, API Gateway

Description: An attacker who intercepts a valid OAuth access token through network eavesdropping, log exposure, or XSS can replay it to authenticate as the victim until expiry. Long token expiry windows grant extended unauthorised access to PHI.

Mitigation: Issue short-lived access tokens (15-minute expiry) with secure refresh token rotation. Bind tokens to client fingerprints where feasible. Implement token revocation endpoints and verify revocation status at the API Gateway on each request. (Ref: OAuth 2.0 Security BCP, RFC 9700)

Threat 8 — Appointment Data Manipulation (Tampering — Medium)

Affected Component: Appointment Service

Description: An attacker exploiting an IDOR vulnerability or a stolen token in the Appointment Service could modify or cancel appointments for other patients, causing operational disruption and potential patient harm if critical care appointments are affected.

Mitigation: Enforce ownership-based access controls — patients can only modify their own appointments. Validate all appointment identifiers against the authenticated user's context server-side. Use opaque, non-sequential appointment IDs. (Ref: OWASP API Security Top 10 API1)

Threat 9 — Broken Access Control on Medical Records (Elevation of Privilege — Critical)**Affected Component:** Medical Records Service

Description: If the Medical Records Service relies solely on the API Gateway for authorisation, a compromised internal service (e.g., a vulnerable Appointment Service) can make direct service-to-service calls to retrieve any patient's records, bypassing patient-level restrictions entirely.

Mitigation: Enforce authorisation at both the API Gateway and each individual microservice. Each service validates caller identity and permissions independently. Adopt a Zero Trust network architecture where no internal service is implicitly trusted. (Ref: NIST 800-207 Zero Trust Architecture, OWASP API Security Top 10 API5)

Threat 10 — SMS Provider Interception (Tampering — Medium)**Affected Component:** Notification Service

Description: If the Notification Service communicates with the SMS provider over a weakly encrypted channel, an attacker can intercept and modify outbound SMS messages, replacing legitimate appointment reminders with phishing links or attacker-controlled content.

Mitigation: Enforce TLS 1.2+ with certificate validation for all provider communications. Use security-audited notification providers. Monitor outbound notification content for anomalies indicative of injection. (Ref: OWASP Top 10 A02, NIST 800-52)

4.4 Attack Chain Analysis

Attack Chain 1: Full PHI Exfiltration via Token Compromise

Step	Threat	Action
1	JWT Secret Exposure (T4)	Attacker discovers JWT signing secret via misconfigured container environment variable.
2	JWT Token Forgery (T1)	Attacker crafts a forged JWT with an admin role claim and arbitrary patient ID.
3	Broken Access Control (T9)	Using the forged token, attacker calls the Medical Records Service directly, bypassing API Gateway restrictions.
4	Impact	All patient PHI records are exfiltrated. Attacker can access, copy, or delete any medical record.

Compounded Impact: The combination of these three threats produces a catastrophic outcome: total unauthorised access to all patient health records. Individual mitigations for each threat are insufficient if any single link in the chain succeeds.

Strategy to Break the Chain: Migrate to **asymmetric JWT signing (RS256)** to eliminate secret exposure risk. Enforce per-service authorisation so the Medical Records Service independently validates token claims. Implement anomaly detection to **flag tokens** with unusual role claims or access patterns.

Attack Chain 2: Diversion Attack During DDoS Response

Step	Threat	Action
1	API Gateway DDoS (T6)	Attacker launches a volumetric DDoS against the API Gateway, consuming security team attention and incident response resources.
2	OAuth Token Replay (T7)	While the security team responds to the DDoS, the attacker replays a captured token to authenticate as a legitimate clinician.
3	PHI Leakage via Notify (T5)	Attacker triggers bulk notification exports containing patient clinical data to an external address.
4	Impact	DDoS serves as distraction while PHI is quietly exfiltrated through the notification pathway.

Compounded Impact: The DDoS amplifies the token replay attack by degrading the organisation's detection and response capabilities. The noise of the DDoS creates an

opportunity for quieter data exfiltration to proceed undetected — exploiting not just technical vulnerabilities but also organisational response limitations.

Strategy to Break the Chain: Deploy automated DDoS mitigation operating independently of the human security team. Implement short-lived tokens to narrow the replay window. Apply DLP controls on the Notification Service to detect and block bulk outbound data transfers.

4.5 Prioritised Remediation Plan

Priority	Mitigation	Justification
1 — Critical	Migrate to RS256 JWT signing; enforce per-service authorisation	Directly breaks Attack Chain 1. Highest severity. Low implementation complexity.
2 — Critical	Deploy secrets management system; rotate all credentials	Eliminates root cause of JWT secret exposure and API key theft. Foundational control.
3 — High	Implement immutable audit logging across all services	Required for HIPAA compliance and forensic capability. Medium implementation effort.
4 — High	Enable API Gateway rate limiting and cloud-based DDoS protection	Prevents Attack Chain 2 initiation. Readily available cloud-native service.
5 — High	Enforce short-lived tokens and implement token revocation	Limits replay attack window. Straightforward change to Identity Service configuration.
6 — Medium	Apply data minimisation to Notification Service payloads	Reduces PHI exposure to third parties. Simple policy-level change with low implementation cost.
7 — Medium	Implement IDOR protection and ownership-based access in Appointment Service	Prevents patient data manipulation. Well-understood remediation pattern.

Remediations are sequenced by two axes: risk severity (criticality of the threatened asset and exploitation likelihood) and implementation feasibility (estimated effort and system disruption). Critical controls with minimal disruption are addressed first; medium-priority controls requiring more architectural change are scheduled subsequently.

5. Conclusion

This assignment has demonstrated the value of threat modelling as a proactive security discipline. Across three progressively complex scenarios, the application of STRIDE within OWASP Threat Dragon revealed a broad spectrum of threats — from straightforward credential attacks in simple authentication systems to sophisticated multi-stage attack chains in microservices healthcare architectures.

Several key lessons emerged. First, trust boundaries are the most critical points of analysis in any DFD: threats are most severe at the intersection of systems with differing trust levels. Second, no single control is sufficient; defence-in-depth — layering complementary controls — is essential in systems handling sensitive data. Third, architectural complexity amplifies risk: the healthcare microservices scenario demonstrated how a vulnerability in one service cascades into systemic compromise when inter-service trust is implicit rather than explicitly verified. Finally, threat modelling delivers maximum value when performed iteratively and collaboratively during system design, rather than as a post-hoc assessment of an already-deployed system.

-END OF REPORT-